

Platform Roadmap: Sep to Dec 2026

Every remaining story to take Groundwork from Round 2 to all six live modules on Vercel

Platform Roadmap: Sep to Dec 2026

Round 2 covered Use Case Readiness only. This roadmap covers everything remaining: hardening Use Case Readiness, finishing Launch Risk and Platform Risk, building the three modules still marked “soon” (Automation Trace, Agent Grounding, UX Audit), and the business layer (cost, KPIs, ROI) needed for AI Management scope.

Eight sprints, Mon 7 Sep through the end of December, all six modules targeted live on Vercel by Sprint 10.

Sprint Calendar

Sprint	Dates	Focus
Sprint 4	Sep 7 to 18	Use Case Readiness hardening
Sprint 5	Sep 21 to Oct 2	Launch Risk completion + Platform Risk remaining issues
Sprint 6	Oct 5 to 16	Automation Trace (new module)
Sprint 7	Oct 19 to 30	Agent Grounding (new module)
Sprint 8	Nov 2 to 13	UX Audit (new module)
Sprint 9	Nov 16 to 27	Cost/KPI/ROI consolidation, cross-module verification
Sprint 10	Nov 30 to Dec 11	Full integration testing, all 6 modules live on Vercel
Sprint 11	Dec 14 to 23	Final polish, documentation, buffer to Dec 31

Sprint 4: Use Case Readiness Hardening

Sep 7 to 18

4.1: POC Documentation & Workflows

User Story: As a BrainyFuture team member, I want documented POC workflows for Use Case Readiness end to end (intake, diagnosis, Fix Router, Judge, export), so that new consultants can onboard onto the tool without needing a walkthrough from the person who built it.

Definition of Done: poc_documentation.md written with step-by-step workflow diagrams; screenshots of each stage; a troubleshooting section for common intake mistakes.

Acceptance Criteria: Given a new consultant with no prior exposure, when they follow the documentation alone, then they can complete a full diagnosis-to-export cycle without external help.

4.2: UI/UX Improvement Pass

User Story: As a consultant presenting a report to a client, I want the report page to be scannable, not a wall of text, so that I can walk a client through findings live without losing them.

Definition of Done: Apply remaining design-critique recommendations, default-collapsed sections beyond what’s done, one-line summaries pulled to the top of every dense card, real bullet lists instead of run-in prose, extended to Evidence Review and Reality Check sections (already landed for Fix Router and Judge).

Acceptance Criteria: Given a first-time viewer, when they land on a report page, then the verdict and top 3 findings are visible without scrolling, and every dense text block has a one-line summary above the full detail.

4.3: Webhook: Automatic Re-Diagnosis on Use Case Change

User Story: As a consultant, I want the diagnostic to automatically refresh when a client's use case details change (new evidence, updated workflow description), so that the report stays current without me manually re-running it every time something changes.

Definition of Done: A webhook endpoint (/api/webhooks/use-case-updated) accepts an update event tied to a report id; on trigger, it reruns the diagnosis with merged new evidence and creates a new, versioned report linked to the original, not overwriting it.

Acceptance Criteria: Given an existing report, when new evidence is submitted via the webhook, then a new report version is created within a defined SLA (under 5 minutes), and the original report remains accessible with a "superseded by" link.

4.4: Alternate Intake: Document Upload

User Story: As a consultant, I want to upload documents (PDFs, exported ticket logs, spreadsheets) directly instead of retyping evidence into a text box, so that intake is faster and evidence is more complete.

Definition of Done: File upload UI added to the intake form; text extraction pipeline for PDF/DOCX/CSV; extracted text auto-populates the evidenceText field, with the original file retained and linked on the report for traceability.

Acceptance Criteria: Given a PDF of support tickets, when uploaded, then the extracted text appears in the evidence field before submission, and the original file is downloadable from the resulting report.

4.5: Escalation Protocol for Significant Judge Findings

User Story: As a consultant, I want a defined process for what happens when LLM-as-Judge flags a "significant" finding, so that a report with unresolved doubts about its own reasoning never reaches a client unreviewed.

Definition of Done: When a Judge review contains any "significant" finding, the report's share/export/download actions require an explicit human acknowledgment step (a confirmation modal naming the specific finding); the acknowledgment is logged with who confirmed and when.

Acceptance Criteria: Given a Judge review with a significant finding, when a consultant tries to download or share the report, then they must explicitly acknowledge the finding before the action proceeds, and that acknowledgment is recorded on the report.

| Sprint 5: Launch Risk & Platform Risk

Sep 21 to Oct 2

5.1: Extend Fix Router + Judge to Launch Risk

User Story: As a consultant using Launch Risk, I want the same Fix Router and LLM-as-Judge features already live on Use Case Readiness, so that every module gives the same depth of support, not just the one that got built first.

Definition of Done: Confirm ReportView/ResultDashboard already renders FixRouterCard and JudgeCard correctly for Launch Risk's report shape; test with a real Launch Risk report; fix any Launch-Risk-specific field gaps in the Judge prompt (its dimension names differ from Use Case Readiness's).

Acceptance Criteria: Given a completed Launch Risk report, when a consultant clicks Find a fix and Check this verdict's reasoning, then both return real, correctly-scoped results referencing Launch Risk's own dimensions, not Use Case Readiness's.

5.2: Platform Risk: Remaining Gaps

User Story: As a consultant using Platform Risk, I want the watchlist cache and alternatives recommendation logic to cover more vendors and stay fresh, so that common vendor checks are fast and not perpetually falling back to slow live research.

Definition of Done: Expand data/watchlist/processed/ coverage to the top 20-30 vendors BrainyFuture clients actually ask about; add a staleness refresh job so cached entries older than 30 days get automatically re-researched instead of silently serving stale data.

Acceptance Criteria: Given a top-20 vendor name, when a diagnosis is requested, then it hits the watchlist cache and returns in under 15 seconds; given a cache entry older than 30 days, when requested, then live research runs instead of

serving stale data.

| Sprint 6: Automation Trace

Oct 5 to 16

6.1: Automation Trace: Core Diagnostic

User Story: As a consultant, I want a diagnostic that checks whether an automation is traceable end to end, so that I can tell a client whether their n8n or plain-English workflow will be debuggable and auditable once running in production.

Definition of Done: Accepts plain-text or n8n JSON workflow input; normalizes into trigger/transform/action steps; runs a traceability checklist (logging present at each step, error handling defined, rollback path exists, ownership assigned); outputs a score and ranked fix-first list, following the same report shape as every other module.

Acceptance Criteria: Given a real n8n workflow export, when submitted, then the diagnostic correctly identifies each trigger/transform/action step and flags any step missing logging or error handling.

6.2: Automation Trace: Non-Technical Readability

User Story: As a consultant presenting to a non-technical client, I want the Automation Trace report to be readable by someone who doesn't know what n8n is, so that I can use it directly in a client meeting, not just internally.

Definition of Done: Report language avoids raw technical jargon in the summary and verdict sections; each flagged step includes a plain-English explanation of why it matters, not just a technical label.

Acceptance Criteria: Given a report generated from a technical workflow, when read by a non-technical reviewer, then they can correctly explain the verdict and top risk without needing translation from a consultant.

| Sprint 7: Agent Grounding

Oct 19 to 30

7.1: Agent Grounding: Core Diagnostic

User Story: As a consultant, I want a diagnostic that checks whether an AI agent's output is actually grounded in real data, so that I can tell a client whether their agent is reliably sourcing its answers or quietly hallucinating.

Definition of Done: Accepts a description of the agent's architecture and data sources (or sample outputs with claimed sources); checks for retrieval-augmented grounding vs. pure model-knowledge answers; flags claims with no traceable source; scores using the same six-dimension pattern as other modules.

Acceptance Criteria: Given a sample agent output with a claimed source, when checked, then the diagnostic correctly flags whether that source actually supports the claim or the claim outruns its evidence, mirroring the same evidence-claim consistency check already proven in LLM-as-Judge.

7.2: Agent Grounding: Reuse the Judge's Evidence-Claim Logic

User Story: As a developer, I want to reuse the evidence-claim consistency checking logic already built and proven in LLM-as-Judge, so that Agent Grounding doesn't reinvent a check that already works.

Definition of Done: Extract the evidence-claim consistency reasoning pattern from lib/prompts/judge.ts into a shared, reusable prompt fragment; Agent Grounding's core prompt imports and applies it directly.

Acceptance Criteria: Given the same test case that caught the Fix Router overclaiming issue in Round 2, when run through Agent Grounding's grounding check, then it correctly flags the same class of issue.

| Sprint 8: UX Audit

Nov 2 to 13

8.1: UX Audit: Core Diagnostic

User Story: As a consultant, I want a diagnostic that checks whether an AI product's interface is trustworthy, so that I can tell a client whether their AI feature's UI sets accurate expectations and doesn't mislead users.

Definition of Done: Accepts a description or screenshot of an AI product's interface; runs a trust-signal checklist (confidence disclosed to the user, errors clearly recoverable, AI-generated content labeled, no dark patterns); reuses the same design-critique reasoning framework applied to Groundwork's own report page earlier this round.

Acceptance Criteria: Given a real interface screenshot, when submitted, then the diagnostic correctly identifies at least the same category of issues a human design critique would (density, hierarchy, honesty of AI-confidence signaling).

8.2: UX Audit: Image Input Support

User Story: As a consultant, I want to submit a screenshot directly, not just a text description, so that the audit reflects the real interface, not my summary of it.

Definition of Done: Image upload support added to UX Audit's intake; vision-capable model call added to the diagnostic pipeline.

Acceptance Criteria: Given a real screenshot upload, when submitted, then the diagnostic report references specific visual elements from that screenshot, not generic UI advice.

| Sprint 9: Business Layer & Cross-Module QA

Nov 16 to 27

9.1: Cost Analysis Documentation

User Story: As BrainyFuture leadership, I want a real cost analysis for running Groundwork across all six modules, so that pricing and internal budgeting decisions are based on real numbers, not guesses.

Definition of Done: cost_analysis.md documenting per-diagnosis compute cost, monthly hosting cost, and total platform operating cost per month at BrainyFuture's actual consultant headcount.

Acceptance Criteria: Given actual usage data from Sprints 4-8, when the cost model is calculated, then it reflects real logged API usage, not estimated averages.

9.2: KPI Framework

User Story: As BrainyFuture leadership, I want a defined set of KPIs tracked across all six modules, so that we can tell if Groundwork is actually working, not just shipped.

Definition of Done: kpi_framework.md defining, per module: adoption rate, average diagnosis turnaround time, Judge agreement rate, Fix Router acceptance rate, report reuse/share rate.

Acceptance Criteria: Every KPI has a stated data source and a target threshold, not just a name.

9.3: Platform-Wide ROI

User Story: As BrainyFuture leadership, I want an ROI calculation for the whole Groundwork platform, not just one client's use case, so that I can justify continued investment in the platform itself.

Definition of Done: platform_roi.md calculating total build + operating cost against consultant hours saved across all 10 consultants at BrainyFuture's realistic engagement volume.

Acceptance Criteria: ROI is calculated at 12 and 36 months, using the same formula and structure as Jane's individual ROI, but at platform scale, with assumptions stated explicitly.

9.4: Cross-Module Verification

User Story: As a developer, I want to confirm Fix Router and LLM-as-Judge work correctly across all six modules, not just the ones tested individually per-sprint, so that there are no integration surprises before general availability.

Definition of Done: A test pass across all six modules' report pages, confirming Fix Router and Judge both render, run, and persist correctly on each.

Acceptance Criteria: Given each of the six modules, when a real diagnosis, Fix Router run, and Judge run are

completed, then all three produce valid, correctly-scoped results with no shared-component bugs.

| Sprint 10: Integration Testing & Vercel Launch

Nov 30 to Dec 11

10.1: Full Regression Pass

User Story: As the BrainyFuture team, I want a full regression test across all six modules before general rollout, so that nothing that worked in isolation breaks once everything is live together.

Definition of Done: Test plan covering all six modules' intake, diagnosis, Fix Router, Judge, and export flows; every test executed and logged with pass/fail.

Acceptance Criteria: 100% of critical-path tests pass before the Vercel production deploy for this phase is approved.

10.2: All Six Modules Live on Vercel

User Story: As a consultant, I want all six diagnostic modules live and accessible in production, so that I can use any of them with any client, not just the two that shipped first.

Definition of Done: lib/modules.ts status updated from "soon" to "live" for Automation Trace, Agent Grounding, and UX Audit once their regression tests pass; production deploy confirmed on Vercel.

Acceptance Criteria: Given the live Groundwork URL, when any of the six modules is selected, then a real diagnosis can be run start to finish in production, not just locally.

| Sprint 11: Final Polish & Buffer

Dec 14 to 23

11.1: Documentation Refresh

User Story: As the BrainyFuture team, I want mvp_documentation.md and the repo structure reference updated to reflect all six live modules, so that the documentation matches what's actually shipped, not just what shipped in Round 2.

Definition of Done: Both documents updated; a new version of the presentation deck reflecting the full six-module platform.

Acceptance Criteria: Every module referenced in the documentation has a corresponding live route, verified by re-opening each.

11.2: Buffer & Contingency

User Story: As the BrainyFuture team, I want time reserved at the end of the roadmap for whatever inevitably slips, so that the Dec 31 deadline doesn't depend on every single sprint going perfectly.

Definition of Done: No new feature work scheduled in this slot; used only for carrying over incomplete items from Sprints 4-10.

Acceptance Criteria: n/a, this is intentionally reserved capacity, not a deliverable.